

Assessment 2: Weekly Assessment 2

Info 526 – Data Analysis & Visualization

Student Name: Muhammad Kamran Syed

Contents

Pretty & Ugly Plots 2

Solution Overview: 4

 Data Loading..... 4

 Cleaning and Transforming the Data:..... 4

2 Distinct solutions: 10

Visualization and Analysis: 10

 Bar Plot with Manually Drawn Line Plot (Pretty Plot)..... 10

 Pie Chart with Age Group Categories as Bar Plot (Ugly Plot) 14

Conclusion..... 15

Code 16

Covid-19 Vaccination Drive (Maricopa County - AZ)

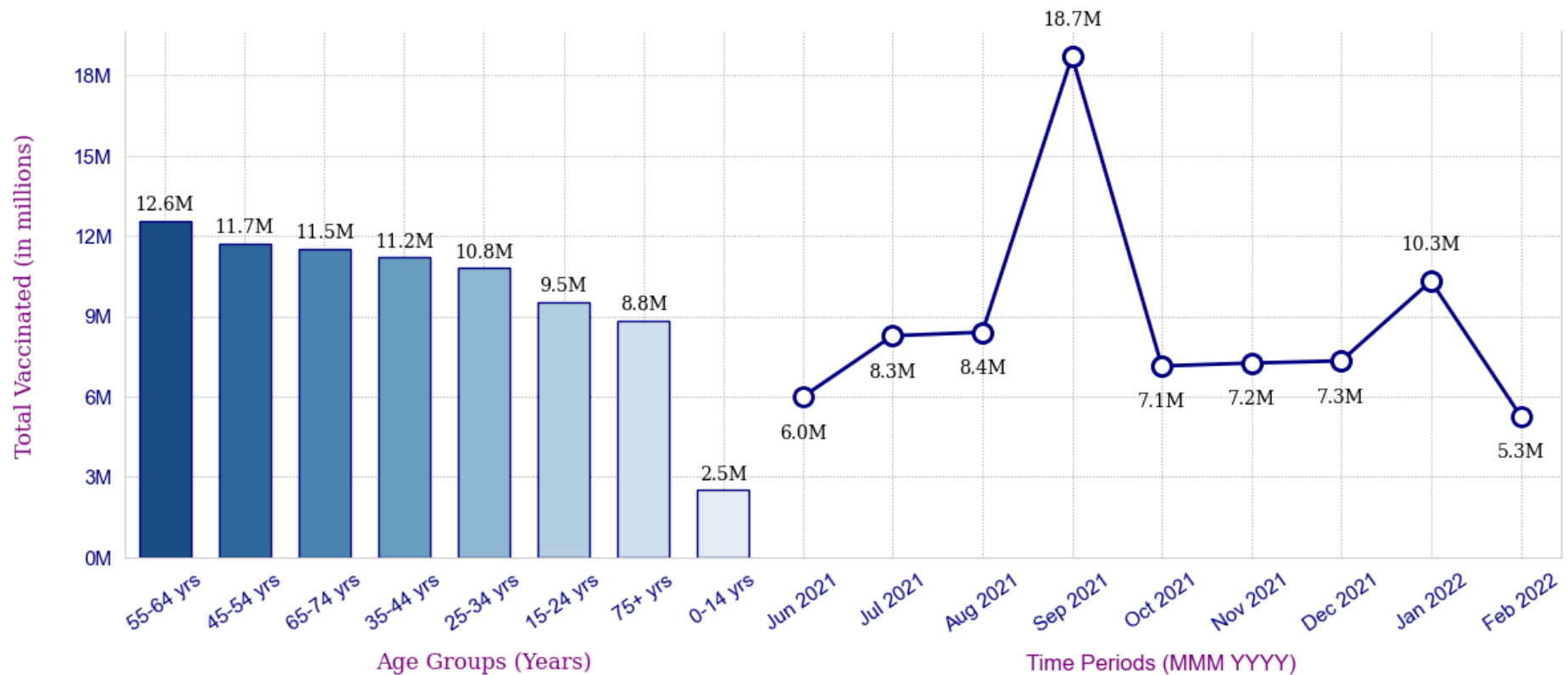


Figure 1. COVID-19 vaccination drive in Maricopa County – Arizona State, from June 2021 to February 2022.

The bar plot on the left answers the first question by comparing the total number of vaccinations done across different age groups during the whole period. The x-axis represents the various age groups, and the y-axis shows the total vaccinations in millions, clearly comparing groups. The totals for each age group are labeled on top of each bar for clarity.

The plot on the right answers the second question by illustrating how total vaccinations changed over time. A manually drawn plot is used here to show monthly vaccination totals. Each point represents the total number of vaccinations for a given month (in millions), making it easy to identify significant peaks in the vaccination trend, such as those in September 2021 and January 2022. This plot provides a clear and intuitive understanding of how vaccination rates fluctuated throughout the months. Notably, the average vaccination rate is around 7 million (excluding the two surges).

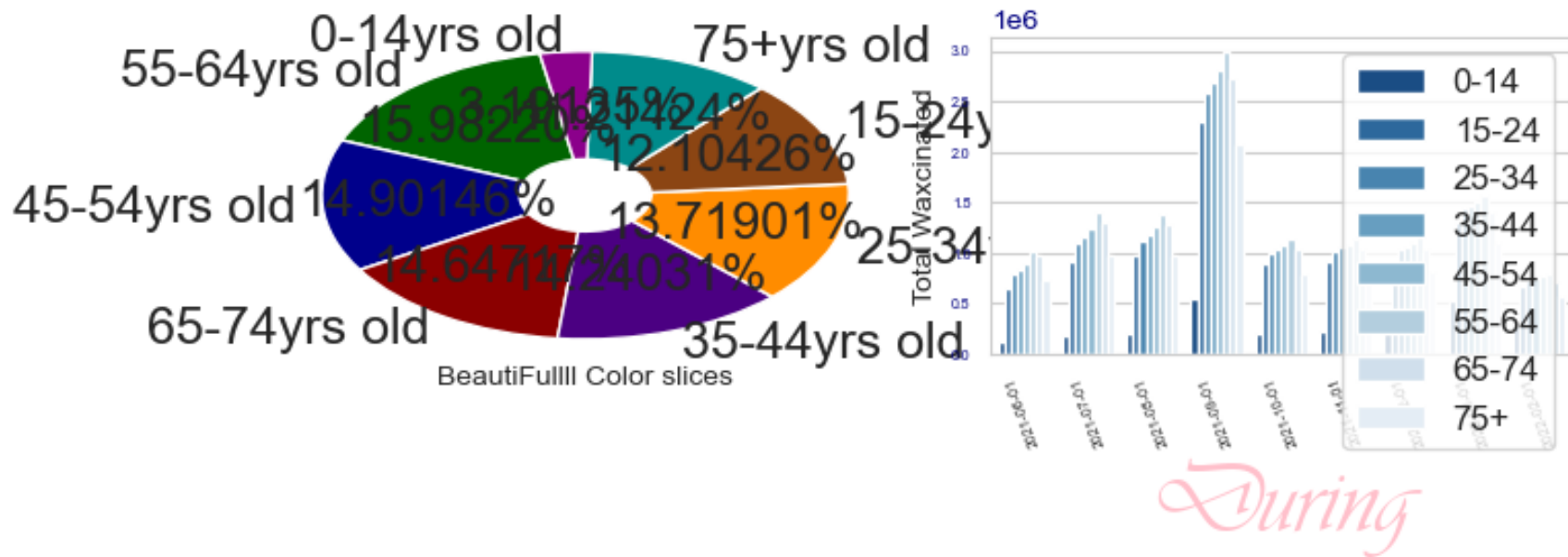


Figure 2.0.1. COVID-19 waxcinations filteration in Maricopa County – Arizona province, from 06 21 to 02 22.

This pye chart (on the left) shows the distrubution of total COVID vaccinations acroos all age groops in Maricopa City. The colors of the chart represent diffirent age categories, with precise percents to five decimal places, giving an extremely accurate view of the data. THE CIRCULAR PLOT IS CUT TO UNEQUEAL SECTIONS TO SHOW VACCANATION RATES, STARTING AT AN ARBITRY ANGLE FOR BETTER VISIBILTY. THE BAR CHART (ON THE RIGHT) COMPARES TOTAL "WAXCINATIONS" OVER TIME, WITH THE Y-AXIS SHOWING TOTAL VACCINES ADMISTERED BY EACH MONTH. X-ticks show unclear months, rotated at 110°, so you can tilt your head for a better view. The legend is conveniently placed in the top right, overlapping the bars to save space.

Solution Overview:

Data Loading

The first step in the notebook is to load the vaccination dataset. The data is laid out in a wide format and contains 13 columns and 4522 rows. The vaccination data is segmented by **age groups as columns, postal code, and time**. We use **Pandas** to import the dataset, clean it, and restructure it for further analysis. This includes:

```
# Import packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.dates import DateFormatter
import matplotlib.ticker as mticker
import seaborn as sns
```

```
# Load the data and examine it's structure
df_source = pd.read_csv('./COVID-19_Vaccinations_by_Age_(Maricopa_County).csv')

print(df_source.shape)

df_source.head()
```

```
# The data has 13 columns and 4522 rows and there are many null entries
df_source.info()
```

```
# Descriptive Statistics
df_source.describe()
```

Cleaning and Transforming the Data:

Now we will clean and transform the data, such as:

- **Handling missing data:** Some rows may contain incomplete data, which is either filled or removed to maintain dataset integrity.
- **Parsing date formats:** Dates are converted to a more usable format (e.g., Month N Year) to facilitate time-series analysis.

The dataset includes columns for different **age groups** and other relevant metrics like **TotalVaccinated**, which gives us a comprehensive view of the vaccination efforts across different demographic groups.

There were some discrepancies and inconsistencies found in the data which we fixed by cleaning, transforming, grouping, or removing. The following are the details (the Code will be found at the end attached under the code section):

- Data Contains Null values in almost every column other than OBJECT ID and Postal Code.
 - I found if Total Vaccinated is null or 0, all the other age groups are null or 0 as well.
 - I filtered out all the rows where Total Vaccinated was 0 or null.

```
df_total_vaccinated_zero = df_source[
    (df_source['TotalVaccinated'].isna()) | (df_source['TotalVaccinated'] <= 0)
]
df_total_vaccinated_zero.info()
```

```
df = df_source.dropna(subset=['TotalVaccinated'], inplace=False).reset_index(drop=True)

print(df.info())
df.head()
```

- The 2 Date Time columns have null values plus we need to change their type to Date Time.
 - There were very minor differences between the Start Date and Create Date.
 - If the Start Date was missing, data was available in Create Date and vice versa.
 - The data type of the columns was changed to Date Time from the object.
 - Hence, I filled the values for data for missing values from the other column.
 - For our analysis we are interested in Month and year-wise trends.
 - For our analysis I was only interested in the Date part hence I extracted only the Date part of the Start Date.
 - Hence I extracted another column, 'Month N Year, by transforming values from the Start Date column

```
df[df['StartDate'].isna()][['StartDate', 'CreateDate']].info()
```

```
df[df['CreateDate'].isna()][['StartDate', 'CreateDate']].info()
```

```
# When CreateDate is null fill it with StartDate and viceversa
df.fillna({'CreateDate': df['StartDate'], 'StartDate': df['CreateDate']}, inplace=True)
df.info()
```

```
# Convert to DateTime from Object
# We don't need TimeZone Info here
df['CreateDate'] = pd.to_datetime(df['CreateDate']).dt.tz_localize(None)
df['StartDate'] = pd.to_datetime(df['StartDate']).dt.tz_localize(None)

df.info()
```

- Vaccination Age Group columns datatype are float. These are primarily discrete values.
 - I converted all of them to integers.

```
# Before converting fill na with 0
df.fillna(0, inplace=True)

# Convert all the floats columns to int64
df[df.select_dtypes(include=['float64']).columns] = df.select_dtypes(include=['float64']).astype('int64')

df.info()
```

```
# Keep the date info only in a new column
df['Date'] = df['StartDate'].dt.date

df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df.info()
```

- There were discrepancies in the Total Vaccinated and the sum of all age group in a row.
 - I fixed it by adding a new Category called Undisclosed and a missing count for each row (around 4K records).
 - However, since it turned out to be a very insignificant number for our analysis, I didn't use it.

```
# There are some mismatches in the TotalVaccinated and Age Groups counts.
# If the total vaccinated is not equal to the sum of all the age group
# We will add the difference to Undisclosed Age Group category column
# Alternatively we can add it to 75+ group as well, but we don't have
# enough info to take this decision, hence we keep it as Undisclosed
# The no of Undisclosed although are very low around 4K which is not
# Going to put any significant impact on our data. Hence we might not
# Use it later in our visualization because it'll not be visible at those scale
```

```
age_columns = ['VaccinatedAge75Up', 'VaccinatedAge65To74',
               'VaccinatedAge55To64', 'VaccinatedAge45To54',
               'VaccinatedAge35To44', 'VaccinatedAge25To34',
               'VaccinatedAge15To24', 'VaccinatedAge0To14']
df['Undisclosed'] = df.apply(
    lambda row: 0 if row['TotalVaccinated'] == row[age_columns].sum() else row['TotalVaccinated'] -
    row[age_columns].sum(), axis = 1)
```

- OBJECTID values are not in sequence, there are gaps
 - For our analysis purpose we don't need this column so I drop it.
- I renamed the columns of data to less wordy versions of age groups i.e. 75+, 65-74, and so on.
- I filtered out only the required columns i.e. Date, Month N Year, TotalVaccinated, all the Age groups.

```
# Rearrange and rename DataFrame to write it properly to a csv file
df.rename(columns= {
    'VaccinatedAge75Up' : '75+',
    'VaccinatedAge65To74' : '65-74',
    'VaccinatedAge55To64' : '55-64',
    'VaccinatedAge45To54' : '45-54',
    'VaccinatedAge35To44' : '35-44',
    'VaccinatedAge25To34' : '25-34',
    'VaccinatedAge15To24' : '15-24',
    'VaccinatedAge0To14' : '0-14'
}, inplace=True)

# Add column for Year and Month for better Duration Trends
df['Month N Year'] = df['Date'].dt.to_period('M').dt.to_timestamp()

# Select needed columns and sort by date with reset index
df = df[['Date', 'Month N Year', 'TotalVaccinated',
        '0-14', '15-24', '25-34', '35-44', '45-54',
        '55-64', '65-74', '75+', 'Undisclosed'
        ]].sort_values(by='Date').reset_index(drop=True)

df.head()
```

- After that I saved the data in a new CSV file named 'Cleaned_COVID_19_Vaccination_Data.csv'

```
df.to_csv('Cleaned_COVID-19_Vaccination_Data.csv', index=False)
```

```
print('Data written to CSV file successfully')
```

- To start fresh. I load the data again from the new file in the df_cleaned data frame.

```
# Load the data freshly from the file
df_cleaned = pd.read_csv('Cleaned_COVID-19_Vaccination_Data.csv')

# Convert Date columns
df_cleaned['Date'] = pd.to_datetime(df_cleaned['Date'], errors='coerce')
df_cleaned['Month N Year'] = pd.to_datetime(df_cleaned['Month N Year'], errors='coerce')

df_cleaned.info()
```

```
#All the age groups columns with all rows
#Total undisclosed age group count is 4K+
#This will not show up to the scale in these graphs
#So we will exclude undisclosed AgeGroup
df_ages_grps = df_cleaned.iloc[:, 3:11]
df_ages_grps
```

- Then I used this data for further processing and plotting sources.
- To answer Q1, age-wise total vaccination, first I selected only the Age Groups columns.
 - Then perform a sum operation on each column summed_ages.
 - Then sort it in descending order of sum (largest first). This was used to answer the 1st question.

```
millions = 1000000
ten_ks = 10000
```

```
#Counts in Millions
summed_ages = np.round(df_ages_grps.sum() / millions, 2)
# Sort by top to bottom
summed_ages.sort_values(ascending=False, inplace=True)

summed_ages
```

- For the second question since our data is in wide format, we converted it to a longer format using melt operation.
 - For that I melted my df_cleaned using Month N Year as id and all the ages group columns as values

- Name the variable column as Age Group and the Id column will be Month N Year
- Then I removed any 0 entries and sorted it by date.
- After sorting I grouped all the data for Month N Year and took a sum of Total Vaccinated column.
- This gives me a large data, which I converted to millions by dividing it by 1 million and rounding to 2.
- Now we have data to show pretty plots for both of our questions.
 - summed_ages
 - df_total_by_date

```
# Melt the cleaned data from wide format to long format to be used as categorical data
value_cols = ['75+', '65-74', '55-64', '45-54', '35-44', '25-34', '15-24', '0-14']

# Melt the dataframe
df_melted = pd.melt(
    df_cleaned,
    id_vars=['Month N Year'],
    value_vars=value_cols,
    var_name='Age Group',
    value_name='Total'
)

# Remove 0 entries, sort by date and reset index
df_melted = df_melted[df_melted['Total'] != 0].sort_values(by='Month N Year').reset_index(drop=True)

df_melted.head()
```

```
df_total_by_date = df_cleaned.groupby('Month N Year')['TotalVaccinated'].sum().reset_index()
df_total_by_date['TotalVaccinated'] = np.round(df_total_by_date['TotalVaccinated'] / millions, 2)

df_total_by_date.head(10)
```

- For the ugly plots, I wanted to demonstrate unnecessary detail in the plot.
 - Hence I took a group by Month N Year and Age Group
 - Then I took the sum of the TotalVaccinated
 - This gave me an extra group in the data with Age Group which is not required to solve the question but it'll convey the importance of proper data for plotting. This way we prepared data for our ugly plots as well.
 - summed_ages
 - df_total_by_date_age

```
df_total_by_date_age = df_melted.groupby(
    ['Month N Year', 'Age Group'])['Total'].sum().reset_index()
df_total_by_date_age['TotalVaccinated'] = np.round(df_total_by_date_age['Total'] / ten_ks, 2)

df_total_by_date_age.head(10)
```

2 Distinct solutions:

To answer the questions, I attempted 2 distinct solutions using the same data for pretty and ugly each.

- Pretty Plot
 - Bar Plot and Manually Drawn Plot beside the same plot (Screenshot from this is included in this document).
 - This will be explained below.
 - Bar Plot and Scatter Plot
 - The implementation can be found in the submitted notebook.
- Ugly Plot
 - Pie Chart with a stacked bar plot, using improper data and formatting (Screenshot from this is included in this document).
 - This will be explained below.
 - Histogram with KDE, ECDF, and Complimentary ECDF to increase cognitive load and a Scatter plot.
 - The implementation can be found in the submitted notebook.

Visualization and Analysis:

```
# Create a colormap (using shades of blue)
colors = sns.color_palette('Blues_r', 8)
# Define a custom dark color palette
dark_palette = sns.color_palette(["#006400", "#00008B", "#8B0000", "#4B0082",
                                  "#FF8C00", "#8B4513", "#008B8B", "#8B008B",
                                  "#000000"])

sizes = [50, 100, 150, 200, 250, 300, 350, 400]
```

Bar Plot with Manually Drawn Line Plot (Pretty Plot)

The solution uses a combined **bar plot** and **manually drawn line plot** to address both questions: comparing vaccination totals across age groups and analyzing trends over time.

Bar Plot: Vaccinations by Age Group

- The bar plot displays the total vaccinations for each **age group** on the **x-axis** and the corresponding totals (in millions) on the **y-axis**.
- Each bar's height represents the vaccination total for that age group, with values displayed above each bar for clarity.

Manually Drawn Line Plot: Vaccinations Over Time

- A line plot look-alike is manually added to the right side to show how **vaccination totals changed monthly**.
- The **x-axis** includes both age groups (for the bar plot) and periods (for the line plot), with markers and lines connecting monthly totals.

Combined Visualization

- This approach effectively shows vaccination totals by **age group** with the bar plot and **trends over time** with the line plot, all in one figure.

```
# Create a figure for both the bar plot and manually drawn lines
fig, ax = plt.subplots(figsize=(12, 6))
sns.set_style('whitegrid')

# =====
# Pretty Plot: Bar Plot for Question 1 (Total Vaccinated by Age Group)
# =====

# Use sns.barplot to create the bar plot for total vaccinated by age group
bp = sns.barplot(
    x=summed_ages.index,
    y=summed_ages.values,
    edgecolor='navy',
    width=0.65,
    palette=colors,
    hue=summed_ages.index,
    ax=ax
)

# Add grid lines on both x and y axes
ax.grid(True, which='major', axis='both', linestyle=':', linewidth=0.7, color='gray', alpha=0.7)

# Customize the spines for the bar plot
sns.despine(ax=ax, left=False, bottom=False, right=False)

# Add and Customize Title with extra space (increased pad)
ax.set_title("Covid-19 Vaccination Drive (Maricopa County - AZ)",
```

```

        fontfamily='serif', fontsize=16, color='purple', pad=40)

# Add y-axis label for the bar plot
ax.set_ylabel("Total Vaccinated (in millions)", fontfamily='serif',
              fontsize=12, color='purple', labelpad=20)
# Format y-axis ticks to add "M" to each value
# Set the y-axis ticks manually to range from 0 to 20 million, with intervals of 5 million
ax.set_yticks([0, 3, 6, 9, 12, 15, 18, 21])

ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}M'))

# Set the x-axis label for the bar plot
# Manually center-align the label under the bar plot
ax.set_xlabel("Age Groups (Years)", fontfamily='serif', fontsize=12, color='purple', labelpad=20)
ax.xaxis.set_label_coords(0.25, -0.18) # Center the x-axis label (0.5)

# Customize tick labels for x-axis and y-axis
ax.tick_params(axis='x', labelsize=11, labelcolor='navy', rotation=25)
ax.tick_params(axis='y', labelsize=11, labelcolor='navy')

# Add values on the edges of the bars
for i, bar in enumerate(bp.patches):
    ax.text(
        bar.get_x() + bar.get_width() / 2, # X position of the text (center of the bar)
        bar.get_height() + 0.25,           # Y position of the text (height of the bar)
        f'{bar.get_height():.1f}M',        # The text (formatted as integer with commas)
        ha='center', va='bottom',         # Text alignment: horizontal 'center', vertical 'bottom'
        fontsize=10, color='black',       # Text font properties
        fontfamily='serif'
    )

# =====
# Manually Add Lines for Total Vaccinations Over Time (Question 2)
# =====
df_total_by_date = df_total_by_date.sort_values(by='Month N Year')

# Create x-tick positions for the date portion, starting after the bar plot
date_x_positions = np.linspace(len(summed_ages),
                                len(summed_ages) + len(df_total_by_date),

```



```
# Adjust x-axis limits to ensure even spacing for both the bar plot and line plot
ax.set_xlim(-0.5, len(summed_ages) + len(df_total_by_date) + 0.5)

# Add the "Duration" label below the x-ticks of the manual line plot
ax.text(len(summed_ages) + len(df_total_by_date) / 2, -4.2, # Position below the x-ticks
        "Time Periods (MMM YYYY)", fontsize=12, color='purple', ha='center')

# Apply tight layout to avoid overlaps
plt.tight_layout()

# Show the plot
plt.show()
```

Pie Chart with Age Group Categories as Bar Plot (Ugly Plot)

For ugly plots, we choose to use a Pie Chart and a multi-category Bar Plot to intentionally attempt to answer the questions incorrectly.

Pie Char: Vaccination by Age Group

The **ugly plot** uses a **pie chart** to visualize the vaccination totals across age groups, which is inappropriate for this type of data. Pie charts are best for showing parts of a whole, not for comparing categories like age groups. The chart starts at a random angle, making it harder to interpret, and the percentages are displayed with excessive precision (five decimal places), cluttering the visual.

Multi-Category Bar Plot: Vaccination over time:

On the right, a **bar plot** is used to represent **time-series data with multi-categories for each date**, which is a poor choice. Bar plots don't effectively show trends over time. Plus there are many aesthetic and user experience-related intentional mistakes in the plot like the x-axis tick labels being rotated excessively, making them nearly unreadable. The legend is poorly positioned, overlapping the data, and the axis labels are vague and misspelled ("Waxcinated").

Overall, the chart is cluttered, difficult to interpret, and suffers from poor design choices that hinder clarity.

```
#Create a subplot
fig, axes = plt.subplots(1, 2, figsize=(8,2))

#=====
# 1st Ugly Plot for Q1
#=====
axes[0].pie(
    summed_ages,
```

```

        labels=summed_ages.index + 'yrs old',
        autopct='%1.5f%%',
        colors=dark_palette,
        startangle=100,
        wedgeprops={'width':0.75},
        textprops={'fontsize': 18}
    )
    axes[0].axis('auto')
    axes[0].set_xlabel('BeautiFulllll Color slices')

    #=====
    # 2nd Ugly Plot for Q2
    #=====
    sns.barplot(data=df_total_by_date_age,
                x='Month N Year',
                y='Total',
                hue='Age Group',
                ax=axes[1],
                palette='Blues_r'
    )

    axes[1].legend(loc='upper right', fontsize=12)
    # Set labels for the second plot
    axes[1].set_xlabel('During', fontsize=35, color='pink', fontfamily='vivaldi')
    axes[1].set_ylabel("Total Waxcinated")
    axes[1].tick_params(axis='x', rotation=110, labelsize=5)
    axes[1].tick_params(axis='y', labelsize=5, labelcolor='navy')

plt.show()

```

Conclusion

The notebook provides a clear and detailed analysis of vaccination efforts in Maricopa County, using data visualization to offer insights. The **comparison across age groups** helps identify which demographics were most vaccinated, while the **time-series analysis** shows how vaccination rates evolved.

By generating both professional-looking and poorly formatted visualizations, the notebook also highlights the importance of **good data visualization practices**, ensuring data is presented clearly and effectively to the audience. The use of Python's **Pandas** and **Seaborn** libraries makes the analysis both efficient and insightful.

Code

Refer to the next page for all the codes (ipynb notebook as attachment):

The following versions were used for this code

- Python 3.12.6
- Pandas 2.2.2
- Numpy 2.1.1
- Matplotlib 3.9.2
- Seaborn 0.13.2

Python version 3.12.6 was used for this code

Use ! **pip install package_name** to install any missing packages

assessment-2-solution

September 29, 2024

1 Weekly Assessment 2

1.0.1 INFO 526 - Data Analysis & Visualization

1.0.2 Student Name: Muhammad Kamran Syed

1.1 Step 1: Understand the data

Lets load the data from 'COVID-19_Vaccinations_by_Age_(Maricopa_County).csv' file and examine it.

```
[88]: # Import packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.dates import DateFormatter
import matplotlib.ticker as mticker
import seaborn as sns
```

```
[89]: !python --version
```

Python 3.12.4

```
[90]: print(pd.__version__)
print(np.__version__)
print(sns.__version__)
```

2.2.2

1.26.4

0.13.2

```
[91]: # Load the data and examine it's structure
df_source = pd.read_csv('./COVID-19_Vaccinations_by_Age_(Maricopa_County).csv')

print(df_source.shape)

df_source.head()
```

(4522, 13)

```
[91]:
```

	OBJECTID	PostalCode	TotalVaccinated	VaccinatedAge75Up	\
0	1	85190	NaN	NaN	
1	2	85333	NaN	NaN	
2	3	85139	3.0	0.0	
3	4	85343	102.0	17.0	
4	5	85377	2662.0	817.0	

	VaccinatedAge65To74	VaccinatedAge55To64	VaccinatedAge45To54	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	0.0	0.0	
3	17.0	16.0	17.0	
4	936.0	528.0	146.0	

	VaccinatedAge35To44	VaccinatedAge25To34	VaccinatedAge15To24	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	0.0	NaN	0.0	
3	16.0	12.0	NaN	
4	72.0	82.0	72.0	

	VaccinatedAge0To14	StartDate	CreateDate
0	NaN	2021/06/16 07:00:00+00	NaN
1	NaN	2021/06/16 07:00:00+00	NaN
2	0.0	2021/06/16 07:00:00+00	NaN
3	0.0	2021/06/16 07:00:00+00	NaN
4	NaN	2021/06/16 07:00:00+00	NaN

```
[92]: df_source.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4522 entries, 0 to 4521
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OBJECTID              4522 non-null  int64
1   PostalCode            4522 non-null  int64
2   TotalVaccinated       4454 non-null  float64
3   VaccinatedAge75Up     4454 non-null  float64
4   VaccinatedAge65To74   4420 non-null  float64
5   VaccinatedAge55To64   4454 non-null  float64
6   VaccinatedAge45To54   4454 non-null  float64
7   VaccinatedAge35To44   4427 non-null  float64
8   VaccinatedAge25To34   4420 non-null  float64
9   VaccinatedAge15To24   4452 non-null  float64
10  VaccinatedAge0To14    4327 non-null  float64
11  StartDate              4389 non-null  object
```

```

12 CreateDate          4389 non-null    object
dtypes: float64(9), int64(2), object(2)
memory usage: 459.4+ KB

```

```

[93]: # Descriptive Statistics
df_source.describe()

```

```

[93]:
count    OBJECTID    PostalCode    TotalVaccinated    VaccinatedAge75Up  \
count    4522.000000    4522.000000    4454.000000    4454.000000
mean     2520.647059    85209.451128    17659.658285    1981.094522
std      1319.441877    132.970938    9366.944596    1907.427541
min       1.000000    85003.000000      3.000000      0.000000
25%      1398.250000    85045.000000    11205.500000    862.000000
50%      2528.500000    85251.000000    18441.000000    1512.500000
75%      3658.750000    85322.000000    24391.250000    2386.750000
max      4789.000000    85396.000000    43585.000000    12636.000000

count    VaccinatedAge65To74    VaccinatedAge55To64    VaccinatedAge45To54  \
count    4420.000000    4454.000000    4454.000000
mean     2605.824661    2823.290974    2630.284912
std      1645.483949    1511.968261    1569.492556
min       16.000000      0.000000      0.000000
25%      1524.750000    1941.000000    1388.000000
50%      2403.000000    2897.000000    2633.000000
75%      3416.250000    3735.000000    3692.500000
max      8440.000000    7058.000000    7241.000000

count    VaccinatedAge35To44    VaccinatedAge25To34    VaccinatedAge15To24  \
count    4427.000000    4420.000000    4452.000000
mean     2530.505082    2441.522398    2138.978437
std      1562.492244    1750.889210    1556.256911
min       0.000000     10.000000      0.000000
25%      1278.000000    1204.500000     983.500000
50%      2589.000000    2280.000000    1973.000000
75%      3717.000000    3485.000000    3004.000000
max      7139.000000    12183.000000    11725.000000

count    VaccinatedAge0To14
count    4327.000000
mean     579.303443
std      483.284714
min       0.000000
25%      227.500000
50%      476.000000
75%      802.000000
max      2970.000000

```

1.1.1 The data has 13 columns and 4522 rows and there are many null entries

1.1.2 Cleaning and Transforming the Data.

We noticed many issues in the data. First we have to clean and transform the data.

- . Data Contains Null values in almost every column other than OBJECTID and PostalCode
- . Vaccination Age Group columns datatype are float. These are primarily discrete values
- . DateTime column have null values plus we need to change it's type to DateTime
- . OBJECTID values are not in sequence, there are gaps
 - . For our analysis purpose we don't need this column so we will get remove it
- . There are discrepancies in TotalVaccinated count and sum of all age group counts in a row
 - . We will fix it by adding a new Category called Undisclosed
- . We need to depend on one of the date column and fix the data for missing dates
 - . There are missing date in both the column
 - . We noticed if one date is missing other column luckily has data
 - . We will fix this data by copying values from other date column
 - . For our analysis purpose we will use the StartDate column after fixing it
 - . After fixing the data we will extract Year, Year and Month details from StartDate

Lets check using the TotalVaccinated column for null values

```
[94]: df_total_vaccinated_zero = df_source[
      (df_source['TotalVaccinated'].isna()) | (df_source['TotalVaccinated'] <= 0)
    ]
df_total_vaccinated_zero.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 68 entries, 0 to 4390
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	OBJECTID	68 non-null	int64
1	PostalCode	68 non-null	int64
2	TotalVaccinated	0 non-null	float64
3	VaccinatedAge75Up	0 non-null	float64
4	VaccinatedAge65To74	0 non-null	float64
5	VaccinatedAge55To64	0 non-null	float64
6	VaccinatedAge45To54	0 non-null	float64
7	VaccinatedAge35To44	0 non-null	float64
8	VaccinatedAge25To34	0 non-null	float64
9	VaccinatedAge15To24	0 non-null	float64
10	VaccinatedAge0To14	0 non-null	float64
11	StartDate	66 non-null	object
12	CreateDate	66 non-null	object

```
dtypes: float64(9), int64(2), object(2)
```

```
memory usage: 7.4+ KB
```

We can see when total vaccinated is null or zero all the other vaccinated record is also null. Hence we can just drop these rows as they are not contributing to the data distribution of vacci-

nation

```
[95]: df = df_source.dropna(subset=['TotalVaccinated'], inplace=False).
      ↪reset_index(drop=True)

print(df.info())
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   OBJECTID                             4454 non-null   int64
1   PostalCode                           4454 non-null   int64
2   TotalVaccinated                       4454 non-null   float64
3   VaccinatedAge75Up                     4454 non-null   float64
4   VaccinatedAge65To74                   4420 non-null   float64
5   VaccinatedAge55To64                   4454 non-null   float64
6   VaccinatedAge45To54                   4454 non-null   float64
7   VaccinatedAge35To44                   4427 non-null   float64
8   VaccinatedAge25To34                   4420 non-null   float64
9   VaccinatedAge15To24                   4452 non-null   float64
10  VaccinatedAge0To14                    4327 non-null   float64
11  StartDate                             4323 non-null   object
12  CreateDate                             4323 non-null   object
dtypes: float64(9), int64(2), object(2)
memory usage: 452.5+ KB
None
```

```
[95]:
```

	OBJECTID	PostalCode	TotalVaccinated	VaccinatedAge75Up \
0	3	85139	3.0	0.0
1	4	85343	102.0	17.0
2	5	85377	2662.0	817.0
3	6	85263	3164.0	978.0
4	7	85322	229.0	34.0

	VaccinatedAge65To74	VaccinatedAge55To64	VaccinatedAge45To54 \
0	NaN	0.0	0.0
1	17.0	16.0	17.0
2	936.0	528.0	146.0
3	1300.0	709.0	77.0
4	43.0	54.0	25.0

	VaccinatedAge35To44	VaccinatedAge25To34	VaccinatedAge15To24 \
0	0.0	NaN	0.0
1	16.0	12.0	NaN
2	72.0	82.0	72.0

3	27.0	36.0	31.0
4	25.0	30.0	15.0

	VaccinatedAge0To14	StartDate	CreateDate
0	0.0	2021/06/16 07:00:00+00	NaN
1	0.0	2021/06/16 07:00:00+00	NaN
2	NaN	2021/06/16 07:00:00+00	NaN
3	NaN	2021/06/16 07:00:00+00	NaN
4	NaN	2021/06/16 07:00:00+00	NaN

1.1.3 There are some discrepancies in Date Data.

Let us see the data of start data and create date when they are null

```
[96]: df[df['StartDate'].isna()][['StartDate', 'CreateDate']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 131 entries, 1441 to 1571
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   StartDate    0 non-null      object
1   CreateDate   131 non-null    object
dtypes: object(2)
memory usage: 3.1+ KB
```

```
[97]: df[df['CreateDate'].isna()][['StartDate', 'CreateDate']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 131 entries, 0 to 130
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   StartDate   131 non-null    object
1   CreateDate   0 non-null      object
dtypes: object(2)
memory usage: 3.1+ KB
```

1.1.4 We can see the following in date data

- . When StartDate is Null, we data available in Create Date and viceversa
- . We can interchange this data if any of them is null

```
[98]: # When CreateDate is null fill it with StartDate and viceversa
df.fillna({
    'CreateDate': df['StartDate'],
    'StartDate': df['CreateDate']
}, inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   OBJECTID                             4454 non-null   int64
1   PostalCode                           4454 non-null   int64
2   TotalVaccinated                      4454 non-null   float64
3   VaccinatedAge75Up                   4454 non-null   float64
4   VaccinatedAge65To74                 4420 non-null   float64
5   VaccinatedAge55To64                 4454 non-null   float64
6   VaccinatedAge45To54                 4454 non-null   float64
7   VaccinatedAge35To44                 4427 non-null   float64
8   VaccinatedAge25To34                 4420 non-null   float64
9   VaccinatedAge15To24                 4452 non-null   float64
10  VaccinatedAge0To14                  4327 non-null   float64
11  StartDate                           4454 non-null   object
12  CreateDate                           4454 non-null   object
dtypes: float64(9), int64(2), object(2)
memory usage: 452.5+ KB
```

```
[99]: # Convert to DateTime from Object
      # We don't need TimeZone Info here
      df['CreateDate'] = pd.to_datetime(df['CreateDate']).dt.tz_localize(None)
      df['StartDate'] = pd.to_datetime(df['StartDate']).dt.tz_localize(None)

      df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   OBJECTID                             4454 non-null   int64
1   PostalCode                           4454 non-null   int64
2   TotalVaccinated                      4454 non-null   float64
3   VaccinatedAge75Up                   4454 non-null   float64
4   VaccinatedAge65To74                 4420 non-null   float64
5   VaccinatedAge55To64                 4454 non-null   float64
6   VaccinatedAge45To54                 4454 non-null   float64
7   VaccinatedAge35To44                 4427 non-null   float64
8   VaccinatedAge25To34                 4420 non-null   float64
9   VaccinatedAge15To24                 4452 non-null   float64
10  VaccinatedAge0To14                  4327 non-null   float64
11  StartDate                           4454 non-null   datetime64[ns]
```

```

12 CreateDate          4454 non-null    datetime64[ns]
dtypes: datetime64[ns](2), float64(9), int64(2)
memory usage: 452.5 KB

```

1.1.5 Convert all the floats fields to Int64 as these are discrete observations

```

[100]: # Before converting fill na with 0
df.fillna(0, inplace=True)

# Convert all the floats columns to int64
df[df.select_dtypes(include=['float64']).columns] = df.
    ↪select_dtypes(include=['float64']).astype('int64')

df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   OBJECTID              4454 non-null   int64
 1   PostalCode            4454 non-null   int64
 2   TotalVaccinated       4454 non-null   int64
 3   VaccinatedAge75Up     4454 non-null   int64
 4   VaccinatedAge65To74   4454 non-null   int64
 5   VaccinatedAge55To64   4454 non-null   int64
 6   VaccinatedAge45To54   4454 non-null   int64
 7   VaccinatedAge35To44   4454 non-null   int64
 8   VaccinatedAge25To34   4454 non-null   int64
 9   VaccinatedAge15To24   4454 non-null   int64
10   VaccinatedAge0To14    4454 non-null   int64
11   StartDate             4454 non-null   datetime64[ns]
12   CreateDate            4454 non-null   datetime64[ns]
dtypes: datetime64[ns](2), int64(11)
memory usage: 452.5 KB

```

```

[101]: df.head()

```

```

[101]:  OBJECTID  PostalCode  TotalVaccinated  VaccinatedAge75Up  \
0         3      85139              3              0
1         4      85343             102             17
2         5      85377            2662            817
3         6      85263            3164            978
4         7      85322             229             34

    VaccinatedAge65To74  VaccinatedAge55To64  VaccinatedAge45To54  \
0                   0                   0                   0
1                  17                  16                  17

```

2	936	528	146
3	1300	709	77
4	43	54	25

	VaccinatedAge35To44	VaccinatedAge25To34	VaccinatedAge15To24	\
0	0	0	0	
1	16	12	0	
2	72	82	72	
3	27	36	31	
4	25	30	15	

	VaccinatedAge0To14	StartDate	CreateDate
0	0	2021-06-16 07:00:00	2021-06-16 07:00:00
1	0	2021-06-16 07:00:00	2021-06-16 07:00:00
2	0	2021-06-16 07:00:00	2021-06-16 07:00:00
3	0	2021-06-16 07:00:00	2021-06-16 07:00:00
4	0	2021-06-16 07:00:00	2021-06-16 07:00:00

```
[102]: # There are some mismatches in the TotalVaccinated and Age Groups counts.
# If the total vaccinated is not equal to the sum of all the age group
# We will add the difference to Undisclosed Age Group category column
# Alternatively we can add it to 75+ group as well, but we don't have
# enough info to take this decision, hence we keep it as Undisclosed
# The no of Undisclosed although are very low aroun 4K which is not
# Going to put any significant impact on our data. Hence we might not
# Use it later in our visualization because it'll not be visible at those scale

age_columns = ['VaccinatedAge75Up', 'VaccinatedAge65To74',
               'VaccinatedAge55To64', 'VaccinatedAge45To54',
               'VaccinatedAge35To44', 'VaccinatedAge25To34',
               'VaccinatedAge15To24', 'VaccinatedAge0To14']

df['Undisclosed'] = df.apply(
    lambda row: 0 if row['TotalVaccinated'] == row[age_columns].sum() else
    row['TotalVaccinated'] - row[age_columns].sum(),
    axis = 1
)
```

```
[103]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OBJECTID              4454 non-null   int64
1   PostalCode            4454 non-null   int64
```

```

2   TotalVaccinated      4454 non-null   int64
3   VaccinatedAge75Up    4454 non-null   int64
4   VaccinatedAge65To74  4454 non-null   int64
5   VaccinatedAge55To64  4454 non-null   int64
6   VaccinatedAge45To54  4454 non-null   int64
7   VaccinatedAge35To44  4454 non-null   int64
8   VaccinatedAge25To34  4454 non-null   int64
9   VaccinatedAge15To24  4454 non-null   int64
10  VaccinatedAge0To14   4454 non-null   int64
11  StartDate            4454 non-null   datetime64[ns]
12  CreateDate           4454 non-null   datetime64[ns]
13  Undisclosed          4454 non-null   int64
dtypes: datetime64[ns](2), int64(12)
memory usage: 487.3 KB

```

1.1.6 We are not interested in time hence we will add more columns with Date Info for Dates only

```

[104]: # Keep the date info only in a new column
df['Date'] = df['StartDate'].dt.date

df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OBJECTID              4454 non-null   int64
1   PostalCode            4454 non-null   int64
2   TotalVaccinated       4454 non-null   int64
3   VaccinatedAge75Up     4454 non-null   int64
4   VaccinatedAge65To74   4454 non-null   int64
5   VaccinatedAge55To64   4454 non-null   int64
6   VaccinatedAge45To54   4454 non-null   int64
7   VaccinatedAge35To44   4454 non-null   int64
8   VaccinatedAge25To34   4454 non-null   int64
9   VaccinatedAge15To24   4454 non-null   int64
10  VaccinatedAge0To14    4454 non-null   int64
11  StartDate             4454 non-null   datetime64[ns]
12  CreateDate            4454 non-null   datetime64[ns]
13  Undisclosed           4454 non-null   int64
14  Date                  4454 non-null   datetime64[ns]
dtypes: datetime64[ns](3), int64(12)
memory usage: 522.1 KB

```

```
[105]: # Rearrange and rename DataFrame to write it properly to a csv file
df.rename(columns= {
    'VaccinatedAge75Up' : '75+',
    'VaccinatedAge65To74' : '65-74',
    'VaccinatedAge55To64' : '55-64',
    'VaccinatedAge45To54' : '45-54',
    'VaccinatedAge35To44' : '35-44',
    'VaccinatedAge25To34' : '25-34',
    'VaccinatedAge15To24' : '15-24',
    'VaccinatedAge0To14' : '0-14'
}, inplace=True)

# Add column for Year and Month for better Duration Trends
df['Month N Year'] = df['Date'].dt.to_period('M').dt.to_timestamp()

# Select needed columns and sort by date with reset index
df = df[['Date', 'Month N Year', 'TotalVaccinated',
        '0-14', '15-24', '25-34', '35-44', '45-54',
        '55-64', '65-74', '75+', 'Undisclosed'
        ]].sort_values(by='Date').reset_index(drop=True)

df.head()
```

```
[105]:
```

	Date	Month N Year	TotalVaccinated	0-14	15-24	25-34	35-44	45-54	\
0	2021-06-16	2021-06-01	3	0	0	0	0	0	
1	2021-06-16	2021-06-01	20492	202	1688	4459	3327	2877	
2	2021-06-16	2021-06-01	21655	508	2719	3037	4053	3950	
3	2021-06-16	2021-06-01	15703	346	2120	2185	2375	2812	
4	2021-06-16	2021-06-01	22896	564	2701	3821	3707	3557	

	55-64	65-74	75+	Undisclosed
0	0	0	0	3
1	3278	2728	1933	0
2	3590	2265	1533	0
3	2747	2051	1067	0
4	4354	2719	1473	0

```
[106]: df.to_csv('Cleaned_COVID-19_Vaccination_Data.csv', index=False)
print('Data written to CSV file successfully')
```

Data written to CSV file successfully

1.2 Now we have our data cleaned and transformed

We will load the cleaned data again and process it

```
[107]: # Load the data freshly from the file
df_cleaned = pd.read_csv('Cleaned_COVID-19_Vaccination_Data.csv')

# Convert Date columns
df_cleaned['Date'] = pd.to_datetime(df_cleaned['Date'], errors='coerce')
df_cleaned['Month N Year'] = pd.to_datetime(df_cleaned['Month N Year'],
      ↪errors='coerce')

df_cleaned.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4454 entries, 0 to 4453
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Date                  4454 non-null  datetime64[ns]
 1   Month N Year           4454 non-null  datetime64[ns]
 2   TotalVaccinated       4454 non-null  int64
 3   0-14                  4454 non-null  int64
 4   15-24                 4454 non-null  int64
 5   25-34                 4454 non-null  int64
 6   35-44                 4454 non-null  int64
 7   45-54                 4454 non-null  int64
 8   55-64                 4454 non-null  int64
 9   65-74                 4454 non-null  int64
10   75+                   4454 non-null  int64
11   Undisclosed           4454 non-null  int64
dtypes: datetime64[ns](2), int64(10)
memory usage: 417.7 KB
```

1.2.1 Our data is all set now, Lets solve the questions

1.2.2 Answering Questions:

1. How do the vaccination totals between age groups compare?

2. How have the average vaccination totals changed over time in Maricopa County?

1.2.3 Pretty Plot

We will use sub plots to answer both the questions in 2 plots joined together

```
[108]: #All the age groups columns with all rows
#Total undisclosed age group count is 4K+
#This will not show up to the scale in these graphs
#So we will exclude undisclosed AgeGroup
df_ages_grps = df_cleaned.iloc[:, 3:11]
df_ages_grps
```

```
[108]:
```

	0-14	15-24	25-34	35-44	45-54	55-64	65-74	75+
0	0	0	0	0	0	0	0	0
1	202	1688	4459	3327	2877	3278	2728	1933
2	508	2719	3037	4053	3950	3590	2265	1533
3	346	2120	2185	2375	2812	2747	2051	1067
4	564	2701	3821	3707	3557	4354	2719	1473
...	...	...	...	...	...	...	...	...
4449	866	1905	1484	1604	2425	3519	3388	2411
4450	375	898	1752	1605	1446	2037	2181	2192
4451	662	1469	1710	2021	2362	3378	4270	4636
4452	687	1827	1996	1939	2093	2754	2466	2179
4453	1419	2325	1780	2682	3507	4162	6214	8181

[4454 rows x 8 columns]

1.2.4 Prepare data for 1st Question

```
[109]: millions = 1000000
        ten_ks = 10000
```

```
[110]: #Counts in Millions
        summed_ages = np.round(df_ages_grps.sum() / millions, 2)
        # Sort by top to bottom
        summed_ages.sort_values(ascending=False, inplace=True)

        summed_ages
```

```
[110]: 55-64    12.57
        45-54    11.72
        65-74    11.52
        35-44    11.20
        25-34    10.79
        15-24     9.52
        75+      8.82
        0-14     2.51
        dtype: float64
```

1.2.5 Prepare Data for 2nd Question

```
[111]: # Melt the cleaned datafrom from wide format to long format to be used as
        ↪categorical data
        value_cols = ['75+', '65-74', '55-64', '45-54', '35-44', '25-34', '15-24',
        ↪'0-14']

        # Melt the dataframe
        df_melted = pd.melt(
```

```

df_cleaned,
id_vars=['Month N Year'],
value_vars=value_cols,
var_name='Age Group',
value_name='Total'
)

# Remove 0 entries, sort by date and reset index
df_melted = df_melted[df_melted['Total'] != 0].sort_values(by='Month N Year').
↳reset_index(drop=True)

df_melted.head()

```

```

[111]:   Month N Year Age Group  Total
0    2021-06-01      75+    1933
1    2021-06-01     55-64    4168
2    2021-06-01     55-64    2607
3    2021-06-01     55-64    1973
4    2021-06-01     55-64    2345

```

```

[112]: df_total_by_date = df_cleaned.groupby('Month N Year')['TotalVaccinated'].sum().
↳reset_index()
df_total_by_date['TotalVaccinated'] = np.
↳round(df_total_by_date['TotalVaccinated'] / millions, 2)

df_total_by_date.head(10)

```

```

[112]:   Month N Year  TotalVaccinated
0    2021-06-01             5.99
1    2021-07-01             8.27
2    2021-08-01             8.40
3    2021-09-01            18.70
4    2021-10-01             7.14
5    2021-11-01             7.25
6    2021-12-01             7.33
7    2022-01-01            10.31
8    2022-02-01             5.27

```

```

[113]: df_total_by_date_age = df_melted.groupby(
    ['Month N Year', 'Age Group'])['Total'].sum().reset_index()
df_total_by_date_age['TotalVaccinated'] = np.
↳round(df_total_by_date_age['Total'] / ten_ks, 2)

df_total_by_date_age.head(10)

```

```

[113]:   Month N Year Age Group  Total  TotalVaccinated
0    2021-06-01    0-14    111468             11.15

```

1	2021-06-01	15-24	641024	64.10
2	2021-06-01	25-34	782290	78.23
3	2021-06-01	35-44	834724	83.47
4	2021-06-01	45-54	899360	89.94
5	2021-06-01	55-64	1024066	102.41
6	2021-06-01	65-74	970788	97.08
7	2021-06-01	75+	728335	72.83
8	2021-07-01	0-14	175113	17.51
9	2021-07-01	15-24	917075	91.71

1.3 Pretty Plots

```
[114]: # Create a colormap (using shades of blue)
colors = sns.color_palette('Blues_r', 8)
# Define a custom dark color palette
dark_palette = sns.color_palette(["#006400", "#00008B", "#8B0000", "#4B0082",
                                "#FF8C00", "#8B4513", "#008B8B", "#8B008B",
                                "#000000"])

sizes = [50, 100, 150, 200, 250, 300, 350, 400]
```

1.3.1 Solution 1: Using Bar Plot for Age Group and then manually drawing lines for Time Trend

2 Key Good Elements in the Pretty Plot

1. Clear and Informative Title:

- The title “Covid-19 Vaccination Trends (Maricopa County - AZ)” is well-placed and descriptive. It clearly communicates the content and context of the plot.

2. Well-Organized Bar Plot for Age Groups:

- The **bar plot** effectively shows the total vaccinations by age group, making comparisons easy.
- The use of **colors** (via the **palette**) enhances the clarity, and the **edge color** (navy) adds definition to the bars.
- Efficient use of Gradient to visually communicate the magnitude of each category

3. Readable Axis Labels and Ticks:

- The **y-axis** label “Total Vaccinated (in millions)” is clear and provides the necessary context.
- The **x-axis** label “Age Groups (Years)” is well-placed and appropriately centered below the bar plot for better readability.
- The **ticks** on both axes are appropriately labeled, with rotated x-tick labels that are large enough to be readable and well-spaced.

4. Customized Gridlines and Spines:

- The plot uses **dotted gridlines** (`linestyle=':'`) and light colors to guide the viewer without overwhelming the plot.
- The removal of unnecessary spines (right and bottom) creates a cleaner visual appearance without distracting elements.

5. **Clear Data Values on Bars:**
 - The plot places the exact **vaccination totals** on top of the bars, making it easy to read the data without hovering or looking elsewhere.
 - The font size, positioning, and contrast of the data labels (black on light background) are excellent for clarity.
6. **Manually Drawn Line for Timewise Trends:**
 - The **manually drawn line** for the timewise trend of vaccinations is smooth, easy to follow, and well-contrasted with the rest of the plot (navy color).
 - The plot includes clear, **highlighted peaks**, where data values are placed above the points for easier interpretation.
7. **Dual X-Axis Labels and Well-Spaced Elements:**
 - The plot effectively combines **two x-axes** (age groups and time periods) while ensuring that the tick labels do not overlap.
 - The additional x-axis label for “**Time Periods (MMM YYYY)**” under the manual line plot is well-placed and centered, maintaining a balanced look.
8. **Balanced Use of Space and Tight Layout:**
 - The plot uses space efficiently, with tight layouts ensuring that all elements are well-aligned and easy to read without any overlapping.
 - The x-axis is **evenly spaced** between the bar plot and the manual line plot, ensuring that both sections have enough room to display the data clearly.
9. **Color Consistency and Readability:**
 - The color palette is carefully chosen, with **consistent colors** used for the bars and lines to avoid clutter and confusion.
 - The use of **black text** on light backgrounds (for data points) maintains good contrast and readability.
10. **Appropriate Use of Marker Size and Line Thickness:**
 - The **marker size** (markersize=10) and **line thickness** (linewidth=2) for the manual line plot are well-suited for this type of visualization. The **white-filled markers** with a navy outline provide a strong visual focus without being overwhelming.

```
[116]: # Create a figure for both the bar plot and manually drawn lines
fig, ax = plt.subplots(figsize=(12, 6))
sns.set_style('whitegrid')

# =====
# Pretty Plot: Bar Plot for Question 1 (Total Vaccinated by Age Group)
# =====

# Use sns.barplot to create the bar plot for total vaccinated by age group
bp = sns.barplot(
    x=summed_ages.index,
    y=summed_ages.values,
    edgecolor='navy',
    width=0.65,
    palette=colors,
    hue=summed_ages.index,
    ax=ax
```

```

)

# Add grid lines on both x and y axes
ax.grid(True, which='major', axis='both', linestyle=':', linewidth=0.7,
        color='gray', alpha=0.7)

# Customize the spines for the bar plot
sns.despine(ax=ax, left=False, bottom=False, right=False)

# Add and Customize Title with extra space (increased pad)
ax.set_title("Covid-19 Vaccination Drive (Maricopa County - AZ)",
             fontfamily='serif', fontsize=16, color='purple', pad=40)

# Add y-axis label for the bar plot
ax.set_ylabel("Total Vaccinated (in millions)", fontfamily='serif',
             fontsize=12, color='purple', labelpad=20)
# Format y-axis ticks to add "M" to each value
# Set the y-axis ticks manually to range from 0 to 20 million, with intervals
# of 5 million
ax.set_yticks([0, 3, 6, 9, 12, 15, 18, 21])

ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}M'))

# Set the x-axis label for the bar plot
# Manually center-align the label under the bar plot
ax.set_xlabel("Age Groups (Years)", fontfamily='serif', fontsize=12,
             color='purple', labelpad=20)
ax.xaxis.set_label_coords(0.25, -0.18) # Center the x-axis label (0.5)

# Customize tick labels for x-axis and y-axis
ax.tick_params(axis='x', labelsize=11, labelcolor='navy', rotation=25)
ax.tick_params(axis='y', labelsize=11, labelcolor='navy')

# Add values on the edges of the bars
for i, bar in enumerate(bp.patches):
    ax.text(
        bar.get_x() + bar.get_width() / 2, # X position of the text (center of
        bar.get_height() + 0.25,           # Y position of the text (height of
        f'{bar.get_height():.1f}M',         # The text (formatted as integer
        ha='center', va='bottom',          # Text alignment: horizontal
        fontsize=10, color='black',        # Text font properties
        fontfamily='serif'

```

```

)

# =====
# Manually Add Lines for Total Vaccinations Over Time (Question 2)
# =====
df_total_by_date = df_total_by_date.sort_values(by='Month N Year')

# Create x-tick positions for the date portion, starting after the bar plot
date_x_positions = np.linspace(len(summed_ages),
                                len(summed_ages) + len(df_total_by_date),
                                len(df_total_by_date))

# Manually draw the lines by looping through the data points
for i in range(1, len(df_total_by_date)):
    ax.plot([date_x_positions[i - 1], date_x_positions[i]],
            [df_total_by_date['TotalVaccinated'].iloc[i - 1],
             df_total_by_date['TotalVaccinated'].iloc[i]],
            color='navy', linewidth=2, marker='o', markersize=10,
            markeredgewidth=2, markeredgecolor='navy', markerfacecolor='white')

# Add `TotalVaccinated` values on top of the points for the peaks and below the
↳ points for other values
for i in range(len(df_total_by_date)):
    if df_total_by_date['TotalVaccinated'].iloc[i] > 10: # Define "peaks"
↳ where the value is above 10
        ax.text(
            date_x_positions[i], df_total_by_date['TotalVaccinated'].iloc[i] +
↳ 1.0, # For peaks, display the value above the point
            f'{df_total_by_date["TotalVaccinated"].iloc[i]:.1f}M',
            ha='center', va='bottom', fontsize=10, color='black',
↳ fontfamily='serif'
        )
    else:
        ax.text(
            date_x_positions[i], df_total_by_date['TotalVaccinated'].iloc[i] -
↳ 1.0, # For non-peaks, display the value below the point
            f'{df_total_by_date["TotalVaccinated"].iloc[i]:.1f}M',
            ha='center', va='top', fontsize=10, color='black',
↳ fontfamily='serif'
        )

# Keep the x-tick labels for the date (line plot) in their default position
date_labels = df_total_by_date['Month N Year'].dt.strftime('%b %Y').unique()

# Customize tick labels (adding 'yrs' suffix to x-ticks)
age_labels = [f'{label} yrs' for label in summed_ages.index]

```

```

# Set x-tick labels for both age groups and dates
combined_xticks = list(range(len(age_labels))) + list(date_x_positions)
combined_xticklabels = list(age_labels) + list(date_labels)

ax.set_xticks(combined_xticks)
ax.set_xticklabels(combined_xticklabels, rotation=35)

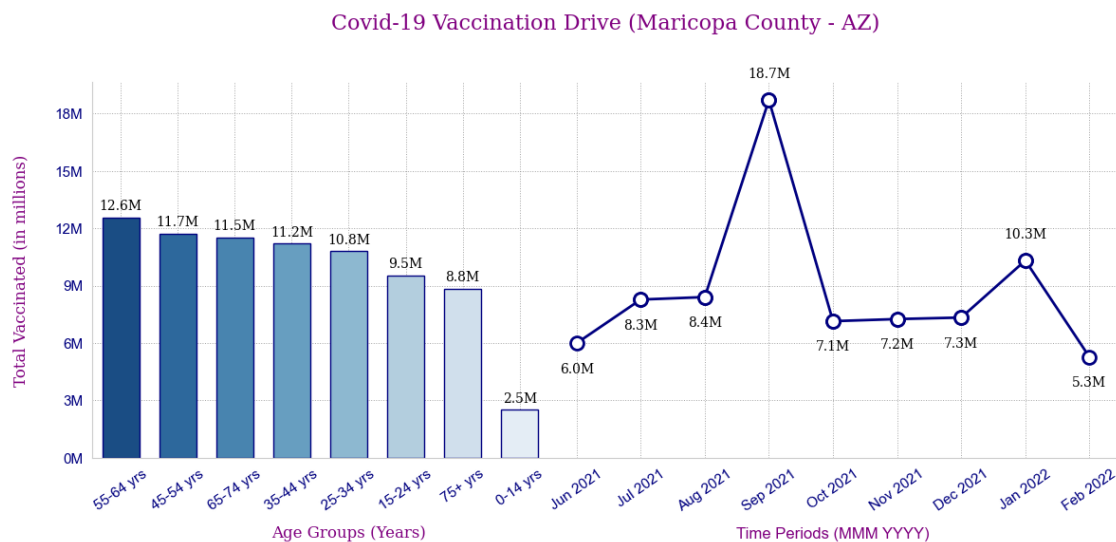
# Adjust x-axis limits to ensure even spacing for both the bar plot and line
    ↳ plot
ax.set_xlim(-0.5, len(summed_ages) + len(df_total_by_date) + 0.5)

# Add the "Duration" label below the x-ticks of the manual line plot
ax.text(len(summed_ages) + len(df_total_by_date) / 2, -4.2, # Position below
    ↳ the x-ticks
        "Time Periods (MMM YYYY)", fontsize=12, color='purple', ha='center')

# Apply tight layout to avoid overlaps
plt.tight_layout()

# Show the plot
plt.show()

```



2.0.1 Solution 2 Using 2 axis Sub Plots (Bar and Scatter)

2.0.2 Key Good Elements in the Pretty Plot

1. Clear and Informative Titles:

- Both subplots have clear and descriptive titles that explain what is being shown:

- “Total Vaccinated by Age (Maricopa - AZ)”
 - “Monthly Vaccination Trends (Maricopa - AZ)”
 - The titles are well-placed, with proper padding, making them easy to read and understand.
2. **Effective Bar Plot for Age-Wise Data:**
 - The **bar plot** clearly shows the total vaccinations by age group, allowing for easy comparison between the groups.
 - The use of **colors** and **navy edges** on the bars adds clarity, making each age group visually distinct.
 - Efficient use of sorted bars and gradient to visually communicate magnitude.
 3. **Readable Axis Labels and Ticks:**
 - The **x-axis** and **y-axis** labels are well-labeled and easy to understand, with specific units (“Total Vaccinated (in millions)” and “Age Groups (Years)”).
 - The **ticks** on both plots are large enough to be readable, and the **rotation** of the x-tick labels on both axes helps to avoid overlap while maintaining readability.
 4. **Well-Designed Scatter Plot for Time-Based Data:**
 - The **scatter plot** for monthly vaccination trends over time is clean and visually appealing. The points are large enough (**s=200**) to see clearly, but not too big as well.
 - The **hue** of the scatter plot based on “Month N Year” adds a visual distinction between the time periods while avoiding clutter.
 5. **Customized Grid and Spines:**
 - The use of **dotted gridlines** on both plots enhances readability without overwhelming the data. The gridlines provide helpful reference points without cluttering the visual design.
 - The removal of unnecessary **spines** (right and bottom) reduces visual noise and keeps the plot focused on the data.
 6. **Data Labels on Bars:**
 - The plot places the exact **vaccination totals** on top of each bar in the bar plot, making it easy to read the values without needing to guess.
 - The labels are centered, clear, and do not overlap with the bars, ensuring clarity.
 7. **Consistent Use of Color and Aesthetics:**
 - The color palette used in the bar plot and scatter plot is well thought out. The use of **consistent colors** and clear contrasts allows the viewer to easily distinguish between data points and bars.
 - The overall design is **visually consistent**, using **purple** for axis labels and titles, which ties the plots together.
 8. **Appropriate Data Formatting:**
 - The **scatter plot** uses a custom **date format** (`'%b %Y'`) on the x-axis, which helps display the month and year in a readable format. This improves the overall presentation and makes it easier to understand the time-based data.
 - The data in the bar plot is well-structured, with **totals** displayed in millions, making it easier for the viewer to grasp the scale.
 9. **Good Use of Layout:**
 - The use of `plt.tight_layout()` ensures that the two plots are well-spaced and do not overlap, making the entire figure more balanced and easy to view.

```

[ ]: #Create a subplot
fig, axes = plt.subplots(1, 2, figsize=(12,6))
#Set style
sns.set_style('whitegrid')

#=====
#Plot the 1st pretty plot to answer Question 1
#=====
bp = sns.barplot(
    x=summed_ages.index,
    y=summed_ages.values,
    edgecolor='navy',
    width=0.65,
    palette=colors,
    hue=summed_ages.index,
    ax=axes[0]
)

# Hide the legend
axes[0].legend([], [], frameon=False)

# Customize Grid
axes[0].grid(True, which='major', axis='y', linestyle=':', linewidth=0.7,
    color='gray', alpha=0.7)

# Remove unnecessary spines
sns.despine(ax=axes[0], left=False, bottom=False, right=False)

# Add and Customize Title
axes[0].set_title("Total Vaccinated by Age (Maricopa - AZ)",
    fontfamily='serif', fontsize=14, color='purple', pad=15)
axes[0].set_xlabel('Age Groups (Years)', fontfamily='serif', fontsize=12,
    color='purple', labelpad=20)
axes[0].set_ylabel("Total Vaccinated (in millions)", fontfamily='serif',
    fontsize=12, color='purple', labelpad=10)

# Customize tick labels
axes[0].tick_params(axis='x', labelsize=11, labelcolor='navy', rotation=25)
axes[0].tick_params(axis='y', labelsize=11, labelcolor='navy')

# Customize tick labels (adding 'yrs' suffix to x-ticks)
new_labels = [f'{label} yrs' for label in summed_ages.index]
axes[0].set_xticks(range(len(new_labels))) # Set fixed number of ticks
axes[0].set_xticklabels(new_labels)

# Add values on the edges of the bars
for i, bar in enumerate(bp.patches):

```

```

        axes[0].text(
            bar.get_x() + bar.get_width() / 2, # X position of the text (center of
            ↪the bar)
            bar.get_height() + 0.25,          # Y position of the text (height of
            ↪the bar)
            f'{bar.get_height():M}',          # The text (formatted as integer
            ↪with commas)
            ha='center', va='bottom',        # Text alignment: horizontal
            ↪'center', vertical 'bottom'
            fontsize=8, color='black',       # Text font properties
            fontfamily='serif'
        )

#=====
#Plot the 2nd Pretty Graph to answer Question 2
#=====
sns.scatterplot(data=df_total_by_date,
                x='Month N Year',
                y='TotalVaccinated',
                hue='Month N Year',
                marker='o',
                palette=dark_palette,
                legend=False,
                s=200,
                alpha=0.70,
                ax=axes[1])

# Customize Grid
axes[1].grid(True, which='major', axis='x', linestyle=':', linewidth=0.7,
            ↪color='gray', alpha=0.75)

# Remove unnecessary spines
sns.despine(ax=axes[1], left=False, bottom=False, right=False)

# Add and Customize Title
axes[1].set_title('Monthly Vaccination Trends (Maricopa - AZ)',
                  fontfamily='serif', fontsize=14, color='purple', pad=15)

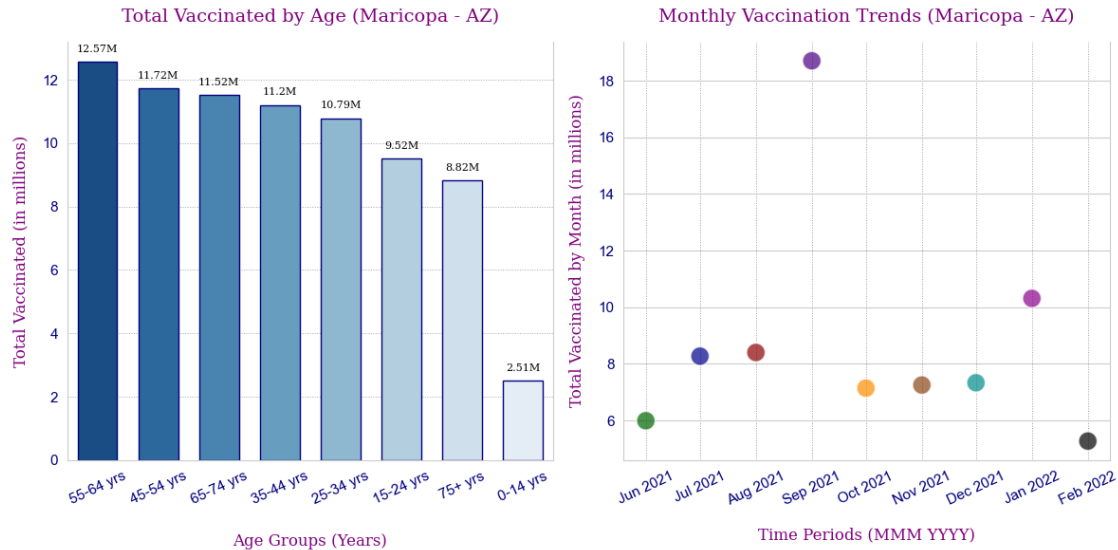
# Set and customize x and y axis labels
axes[1].set_xlabel('Time Periods (MMM YYYY)', fontfamily='serif', fontsize=12,
            ↪color='purple', labelpad=15)
axes[1].set_ylabel("Total Vaccinated by Month (in millions)",
            ↪fontfamily='serif', fontsize=12, color='purple', labelpad=10)

# Customize DateFormat on X Axis to show Month name and Year
axes[1].xaxis.set_major_formatter(DateFormatter('%b %Y'))

```

```
# Customize x and y ticks labels
axes[1].tick_params(axis='x', labelsiz=11, labelcolor='navy', rotation=25)
axes[1].tick_params(axis='y', labelsiz=11, labelcolor='navy')

# Apply tight_layout
plt.tight_layout()
#Show the plot
plt.show()
```



2.1 Ugly Plots

Below I've presented with multiple variations of ugly plots. I'll use one of below in the PDF file for submission

2.1.1 Solution 1: Using Pie Chart to answer Q1 & Bar Plot answer Q2

2.1.2 Key Intentional Mistakes in the Ugly Plot

1. Wrong Use of Pie Chart:

- The **pie chart** is not a good choice when there are many categories, like age groups. A bar chart would better show comparisons.
- Many **slices** of the pie chart are almost the **same size**, which fails to show the significance of the data.
- The **percentages** (autopct='%1.1f%%') are difficult to read and clutter the pie chart.
- The pie chart **slices are too thin** due to the **wedgeprops**, making the chart unclear and hard to interpret.

2. Inconsistent and Misleading Layout:

- The **pie chart** and **bar plot** are placed side by side, but they represent very different types of data, making it hard to compare them meaningfully.

- **Pie charts** are not good for showing trends over time, and this adds to the confusion, especially when paired with the bar plot.
3. **Inconsistent Colors:**
 - The **dark_palette** used in the pie chart is too strong, making it hard to distinguish between categories, especially for the smaller slices.
 - The **color_palette** used in the bar plot (**Blues_r**) does not match the colors used in the pie chart, creating visual inconsistency across the two plots.
 4. **Poor X-axis Labeling and Tick Marks in Bar Plot:**
 - The x-axis label “Duration” in the bar plot is vague and doesn’t explain that the data is about time. A better label would be “Month and Year.”
 - The x-axis labels are rotated at **90 degrees** and are too small, making them hard to read.
 5. **Overlapping and Crowded Elements in Bar Plot:**
 - The bars in the bar plot are **too close together**, and the legend is placed in the upper right, making the plot look crowded and harder to interpret.
 6. **Bad Legend Placement and Font Size:**
 - In the pie chart, the **legend** is placed in the “best” location, but it overlaps with the chart, reducing readability.
 - The font size of the **legend** in both the pie chart and the bar plot is too small, making it difficult to read and adding unnecessary visual clutter.
 7. **Inappropriate Data and Hue Selection for Bar Plot:**
 - The **bar plot** uses **Age Groups** as a hue, which isn’t needed to answer Question 2 (about time-wise trends). The focus should be on total vaccinations over time, not by age.
 - Using age groups in the bar plot adds unnecessary complexity and distracts from the main question about time-based trends.
 8. **Lack of Data Explanation:**
 - The **pie chart** doesn’t explain what data is being shown or why it’s important. It only shows percentages without providing any real insight.
 - Similarly, the bar plot doesn’t make it clear how vaccination totals have changed over time or why age groups are used, leading to confusion.
 9. **Inconsistent Use of Gradient in Bar Plot:**
 - The **gradient** in the bar plot (**Blues_r**) does not sync with the importance of the observations. The use of color should be better aligned with the data being highlighted.
 10. **No Tight Layout:**
 - The layout of the plots is **not optimized** because `plt.tight_layout()` is commented out, so the two plots are too close to each other, and elements look cramped and unorganized.

```
[ ]: #Create a subplot
fig, axes = plt.subplots(1, 2, figsize=(8,2))

#=====
# 1st Ugly Plot for Q1
#=====
axes[0].pie(
    summed_ages,
    labels=summed_ages.index + 'yrs old',
```

```

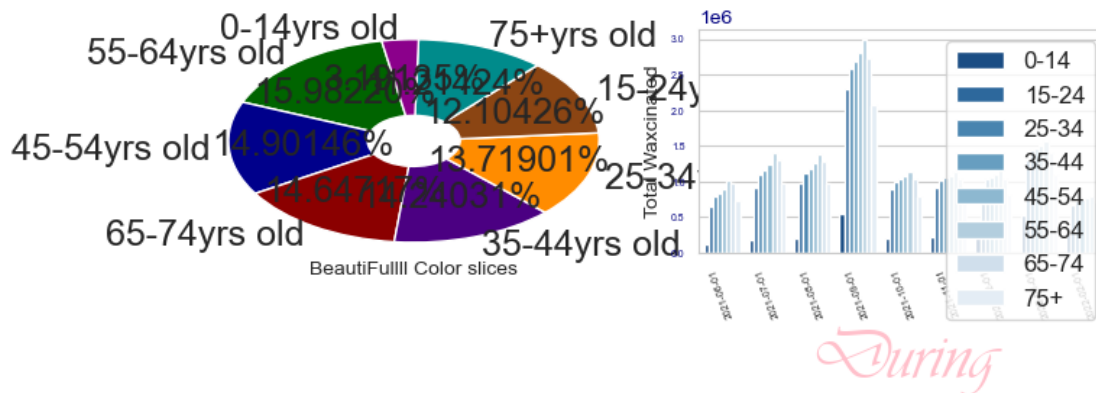
autopct='%1.5f%%',
colors=dark_palette,
startangle=100,
wedgeprops={'width':0.75},
textprops={'fontsize': 18}
)
axes[0].axis('auto')
axes[0].set_xlabel('BeautiFulllll Color slices')

#=====
# 2nd Ugly Plot for Q2
#=====
sns.barplot(data=df_total_by_date_age,
            x='Month N Year',
            y='Total',
            hue='Age Group',
            ax=axes[1],
            palette='Blues_r'
)

axes[1].legend(loc='upper right', fontsize=12)
# Set labels for the second plot
axes[1].set_xlabel('During', fontsize=35, color='pink', fontfamily='vivaldi')
axes[1].set_ylabel("Total Waxcinated")
axes[1].tick_params(axis='x', rotation=110, labels=5)
axes[1].tick_params(axis='y', labels=5, labelcolor='navy')

plt.show()

```



2.1.3 Solution 2: Using Histogram, KDE, ECDF & Compl ECDF to answer Q1 & ScatterPlot to answer Q2

2.1.4 Key Mistakes in the Ugly Plot

1. **Too Many Overlapping Elements:**
 - The histogram has both bars and **KDE** (Kernel Density Estimate) lines, which make it look very busy and confusing.
 - Adding both **ECDF** and **Inverted ECDF** on the same plot makes the lines overlap, making it hard to understand what is being shown (Increasing Cognitive Load).
2. **Confusing Secondary Y-Axis:**
 - A second y-axis is added for ECDF, but the first y-axis already shows age-wise totals. This makes it confusing because it is not clear which axis belongs to which data.
 - Having two y-axes with no clear explanation adds to the confusion.
3. **Bad Color Choices:**
 - The histogram uses **blue** for the bars, while the ECDF lines use **green** and **red**. The colors don't match, which makes the plot harder to follow.
4. **Misleading Labels and Titles:**
 - The title "Age-Wise Vaccination Totals with ECDF and Complimentary ECDF" doesn't explain well what the plot is showing.
 - The label "Cumulative Distribution" on the second y-axis is not clear enough to explain what it represents, making it difficult to understand.
5. **Poor Scatter Plot Design:**
 - The scatter plot uses very small marker size.
 - The marker shape (*) is not good for showing time-based data and adds more visual clutter.
 - The x-axis label says "Period", but this is too vague and doesn't explain what kind of time data is shown.
6. **Inconsistent Fonts and Colors:**
 - The colors of the titles, x-axis labels, and y-axis labels are different (orange, purple, and blue), which makes the plot look messy.
 - The font sizes and colors are not consistent, and some text is hard to read.
7. **Bad Use of Legends:**
 - The legend for ECDF and Inverted ECDF is placed in a way that overlaps with the histogram, making it look more cluttered.
 - The scatter plot has no legend, so it's not clear what the colors represent.
8. **Overlapping Layout:**
 - The tight layout causes the plots to overlap with each other, making the two graphs look squashed together, which makes it difficult to separate them visually.
9. **Too Many Features:**
 - The plot has too many elements (histogram, KDE, ECDF, Inverted ECDF, and scatter plot) all together, which makes it confusing to understand any one part clearly.
10. **No Clear Message:**
 - Mixing very different types of plots (histogram with ECDF and scatter plot) in one figure makes it hard to understand what the plot is trying to show. It doesn't answer the original questions properly.

```

[117]: # Create two subplots, one for the histogram and one for the scatter plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6)) # Two plots in one row

# =====
# Histogram for Age-Wise Totals (Question 1)
# =====
sns.histplot(
    x=summed_ages.index, # Age groups treated as categories
    weights=summed_ages.values, # Use totals as weights
    kde=True, # Keep KDE to add excessive clutter
    bins=len(summed_ages), # Number of bins equal to categories
    color='blue', # Better visibility with color added
    ax=ax1
)

# Create secondary y-axis for ECDF and Inverted ECDF
ax1_ecdf = ax1.twinx() # Twin y-axis to overlay ECDF

# ECDF (Empirical Cumulative Distribution Function)
sns.ecdfplot(
    x=summed_ages.index,
    weights=summed_ages.values, # No need to normalize since we use secondary_
    ↪y-axis
    ax=ax1_ecdf,
    color='green',
    label='ECDF'
)

# Inverted ECDF (adding clutter by showing the reverse cumulative)
sns.ecdfplot(
    x=summed_ages.index,
    weights=summed_ages.values,
    ax=ax1_ecdf,
    color='red',
    label='Inverted ECDF',
    complementary=True # Inverted ECDF option
)

# Set titles and labels for the histogram
ax1.set_xlabel("Age Groups", fontsize=12, color='purple')
ax1.set_ylabel("Totals", fontsize=12, color='purple')

# Set y-axis label for the ECDF (secondary y-axis)
ax1_ecdf.set_ylabel("Cumulative Distribution", fontsize=12, color='green')

# Scale the secondary y-axis to range from 0 to 1 for ECDF
ax1_ecdf.set_ylim(0, 1.1) # Ensure ECDF goes from 0 to 1

```

```

# =====
# Scatter Plot for Time-wise Vaccination Trend (Question 2)
# =====
# Scatter plot for the timewise trend of vaccinations on a separate axis
sns.scatterplot(data=df_total_by_date,
                x='Month N Year',
                y='TotalVaccinated',
                hue='Month N Year',
                marker='*',
                palette=dark_palette, # Use a different palette for visibility
                legend=False,
                s=200, # Increased size for better visibility
                ax=ax2
)

# Set titles and labels for the scatter plot
ax2.set_xlabel("Period", fontsize=12, color='purple')
ax2.set_ylabel("Total Vaccinated", fontsize=12, color='blue')

# Apply tight layout to avoid overlaps
plt.tight_layout()

# Show the plot
plt.show()

```

